5.1 Python's Sequence Types

In this chapter, we explore Python's various "sequence" classes, namely the built-in **list**, **tuple**, and **str** classes. There is significant commonality between these classes, most notably: each supports indexing to access an individual element of a sequence, using a syntax such as seq[k], and each uses a low-level concept known as an ***array*** to represent the sequence. However, there are significant differences in the abstractions that these classes represent, and in the way that instances of these classes are represented internally by Python. Because these classes are used so widely in Python programs, and because they will become building blocks upon which we will develop more complex data structures, it is imperative that we establish a clear understanding of both the public behavior and inner workings of these classes.

Public Behaviors

A proper understanding of the outward semantics for a class is a necessity for a good programmer. While the basic usage of lists, strings, and tuples may seem straightforward, there are several important subtleties regarding the behaviors associated with these classes (such as what it means to make a copy of a sequence, or to take a slice of a sequence). Having a misunderstanding of a behavior can easily lead to inadvertent bugs in a program. Therefore, we establish an accurate mental model for each of these classes. These images will help when exploring more advanced usage, such as representing a multidimensional data set as a list of lists.

Implementation Details

A focus on the internal implementations of these classes seems to go against our stated principles of object-oriented programming. In Section 2.1.2, we emphasized the principle of ***encapsulation***, noting that the user of a class need not know about the internal details of the implementation. While it is true that one only needs to understand the syntax and semantics of a class's public interface in order to be able to write legal and correct code that uses instances of the class, the efficiency of a program depends greatly on the efficiency of the components upon which it relies.

Asymptotic and Experimental Analyses

In describing the efficiency of various operations for Python's sequence classes, we will rely on the formal ***asymptotic analysis*** notations established in Chapter 3. We will also perform experimental analyses of the primary operations to provide empirical evidence that is consistent with the more theoretical asymptotic analyses.